A high-performance Delta lake for Drishti on AWS

Past 200 TB, and still fast: the lake in S3, the scripts, the keys, the partitions and every optimisation

Status: scripts in deploy/aws/, images in deploy/docker/ecs/, every script tested with --help and --dry-run; first real run in a development account pending
Source of truth: docs/architecture/AWS_DELTA_LAKE.md (Drishti 2.6.5)

Drishti · Any data. Any domain. One grammar.

Copyright © 2026 Ashutosh Sinha. All rights reserved. Proprietary and confidential.

What this deck covers

1 The picture: from the sources to the screen
2 Why S3 and not EFS for the lake
3 Built and planned
4 First-time setup, step by step (steps 0–11)
5 The layout: buckets, prefixes, tables, partitions
6 Keys: the id, promoted columns, statistics
7 Files: sizes, row groups, sorting
8 Loading: daily load, backfill, restatement
9 Maintenance: checkpoints, retention, vacuum, tiers
10 Caches: EFS or local NVMe
11 Staying fast past 200 TB
12 Sizing and cost
13 Runbooks
14 The scripts
15 Limits, what comes next, decisions

The one rule: the lake lives in Amazon S3

read this first

Every table's Parquet data files AND its _delta_log are in S3. Nothing of the lake is on EFS.
EFS holds only what a node must find again after a restart: configuration, users and tokens, Drishti*net state, and the caches that should be warm after a restart (or local NVMe for those caches).

- **S3 (the lake):** `s3://<org>-drishti-<env>-lake/lake/<domain>/<table>/` with `business_date=…/PRODUCT_TYPE=…/part-….snappy.parquet` and `_delta_log/`
- **S3 (landing):** the sources' dumps, `<date>/<source>/*.jsonl.gz`, kept 30 days (`LANDING_KEEP_DAYS`), then expired
- **S3 (logs):** S3 access logs, SSE-S3, expire after 400 days
- **EFS (state):** `/drishti/<env>/<node>/opt/drishti/{data,config,sutras}`, one access point per node per path, identical path inside the container
- **EFS (loader scratch):** one day of one table for the minutes of a load, then removed; plus the per-table lock files
- **Never on EFS:** lake data, `_delta_log`

Status: what exists today, and how it was tested

Piece	Status	Where
Reading Delta tables in S3 (native engine, no Hadoop)	**built**	`drishti-deltalake`, `drishti-lakeio`, `plugins/drishti-plugin-delta`
`lake build` (JSON export → Delta table on local disk or EFS)	**built**	`tools/lake/build.py` + Drishti's Java writer
`lake publish` (a built table → ONE commit in S3)	**built (new in this work)**	`tools/lake/publish.py`, `tools/lake/test_publish.py`
`lake maintain` on `s3://` (retention, checkpoint, vacuum)	**built**	`tools/lake/maintain.py` with `--force-foreign`
AWS scripts: buckets, IAM, conditional-write check, lifecycle, alarms, report, load, backfill, maintain, EFS, Cloud Map, register	**scripts**, `--dry-run` tested; first real run pending	`deploy/aws/`
ECS images: server, console, loader + entry points	**files**, not yet built in CI	`deploy/docker/ecs/`

Tests: `tools/test_deploy_aws.py` runs every script with `--help` and with `--dry-run` against a fake settings file and an `aws` that fails the test if called; renders all 11 JSON templates; checks the lifecycle rules and that `deploy/aws/env` is git-ignored. `test_publish.py`: 5 tests.

01
The picture

From the source systems' dumps to a trader's screen: landing bucket → loader task → the lake in S3 → Drishti servers → EFS for state

The picture: the whole flow on one page

Sources

- ENDUR, CALYPSO, MUREX, reference
- dumps: JSON / JSON Lines / .gz

S3 · <org>-drishti-<env>-landing

- `2026-10-20/endur/*.jsonl.gz`, `…/calypso/`, `…/murex/`
- kept LANDING_KEEP_DAYS (30), then expired

EventBridge Scheduler

- 22:30 New York, Monday to Friday
- runs the ECS task `loader` (image drishti-loader-ecs)

ECS task · loader · load-day.sh, per table, under the table's lock (EFS)

- 1 `aws s3 sync` landing → `/opt/drishti/data/lake-build/<run>/in/<table>/` (EFS scratch) 2 `lake build` → a scratch Delta table: promoted columns, sorted by id, row groups 1 MB
- 3 `aws s3 sync` scratch data files → `s3://…-lake/lake/<domain>/<table>/business_date=…/` 4 `lake publish` ONE Delta commit on S3 (conditional PUT) 5 metrics `Drishti/Lake`

S3 · <org>-drishti-<env>-lake (THE LAKE)

- `lake/trading/trade_endur/_delta_log/` commits, checkpoints, `_last_checkpoint`
- `…/business_date=2026-10-20/PRODUCT_TYPE=IRS_FIXFLOAT/part-….snappy.parquet`
- versioning · SSE-KMS · only through the VPC endpoint · tiers by business date: Standard → Standard-IA → Glacier IR

ranged GETs

ECS services drishti-a, -b, -c (server + console)

- Delta connector, native engine, root `s3://…-lake/lake` (DRISHTI_DELTA_ROOT)
- heap: file lists, id maps, column sets, documents, footers
- Drishti*net between task IPs; seeds by Cloud Map names

ECS task · maintenance (nightly)

- checkpoint, retention, vacuum on the lake; lifecycle rules for the coming months

EFS (Regional, encrypted, Elastic throughput)

- per-node state and caches; loader scratch and table locks; NO lake data, NO _delta_log

The picture, in words: each box

Box	What it is	Key facts
Sources	ENDUR, CALYPSO, MUREX, reference systems	send dumps as JSON, JSON Lines or gzipped; one folder per date and source
Landing bucket	`<org>-drishti-<env>-landing`	`<date>/<source>/…`; dumps expire after 30 days (`LANDING_KEEP_DAYS`); same protections as the lake
Scheduler	EventBridge Scheduler	`LOAD_SCHEDULE` 22:30 in `LOAD_SCHEDULE_ZONE` (New York), Mon–Fri; also `-maintain` and `-report` schedules
Loader task	ECS task `drishti-<env>-loader`, image `drishti-loader-ecs`	runs `load-day.sh`: sync → build → sync → publish → metrics, per table, under the table's EFS lock; scratch always removed
The lake	`<org>-drishti-<env>-lake`	Delta tables: Parquet + `_delta_log`; versioning, SSE-KMS, VPC endpoint only, tiers by business date
Server nodes	ECS services `drishti-a/-b/-c`	server + console per task; reads the lake by ranged GETs; caches in heap and on disk; Drishti*net between task IPs
Maintenance task	ECS task, nightly (`MAINTAIN_SCHEDULE`)	`lake maintain` (retention, checkpoint, vacuum) per table under its lock, then `setup-lifecycle.sh`
EFS	Regional, encrypted, Elastic throughput	`/drishti/<env>/<node>/opt/drishti/{data,config,sutras}`; loader scratch and locks; never lake data

02
Why S3 and not EFS for the lake

Cost, request rate, latency, durability, tiers, other readers and writers, compared at 200 TB

Why S3 and not EFS: the comparison

	Amazon S3	Amazon EFS
Cost of 200 TB (list prices, one Region)	about **USD 1,400 a month** tiered by business date; USD 4,600 all in Standard	**USD 60,000 a month** in EFS Standard; USD 3,200 in Infrequent Access plus a charge on every byte read and tens of ms added per read
Request rate	at least **3,500 writes and 5,500 reads a second per prefix**; every `business_date=…/PRODUCT_TYPE=…/` folder is a prefix, so the rate grows with the folders	one file system's limits (Elastic: GB/s across all clients, IOPS per file system), shared by every node, loader and maintenance job
Latency of a single read	10–30 ms to the first byte in-Region; Drishti reads one row group	about 1 ms for hot data in Standard; tens of ms in IA
Durability	eleven nines, ≥ 3 AZs, versioning, Cross-Region Replication	Regional (three zones); backup by AWS Backup
Tiers	Standard → Standard-IA → Glacier Instant Retrieval (still ms); per prefix and date by lifecycle rules	Standard → IA → Archive by last access only
Other readers	Athena, Trino, Spark, EMR, Glue read the same Delta tables	only something that mounts the file system
Writers	conditional writes make concurrent commits safe	a hard link on a POSIX file system

At 200 TB+ EFS costs **10–40×** more and shares one throughput budget. Drishti's reads are built for object storage: one day's files, one row group of one file, by byte ranges. **So the lake is in S3; EFS keeps the small, hot, per-node state.**

03
Built and planned

What runs today, what is scripted but not yet run in AWS, and what SUPER_SCALE steps S1–S8 will add

Built and planned (1/2): built and scripted

Capability	Status	Where
Reading Delta tables in S3: file lists from the last checkpoint, id maps, one row group per read, footers cached, documents cached on disk	built	`drishti-deltalake`, `drishti-lakeio`, `plugins/drishti-plugin-delta`; DELTA_CONNECTOR.md
`lake build`: a JSON export into a Delta table on local disk or EFS: promoted columns, partitions, sorted by id per partition that fits a batch	built	`tools/lake/build.py`, the Java writer in the server jar
`lake publish`: a table `lake build` wrote, committed into an S3 table as one commit (S3 conditional writes)	built (this release)	`tools/lake/publish.py`, `test_publish.py`
`lake maintain` on an `s3://` root: retention, checkpoint and log cleanup, vacuum	built	`tools/lake/maintain.py` (`--force-foreign`)
Buckets, IAM, conditional-write check, lifecycle tiers, alarms, report, load, backfill, maintenance, EFS access points, Cloud Map, task/service definitions, schedules	scripts (`--dry-run` tested)	`deploy/aws/`
ECS images (server, console, loader) and entry points	files	`deploy/docker/ecs/`
Drishti*net shards by table; replicas send one entity to the same node by a hash of its id	built	DRISHTI_NET.md §7, §10

Built and planned (2/2): planned, by SUPER_SCALE step

Step	What it adds	Why it matters past 200 TB
S1	Range maps from the log's per-file id statistics; Parquet Bloom filters on `id`	find an id without building a day's id map
S2	Tables by year (`root: …/{year}`), closing a year	a table's checkpoint stops growing with seven years of live files
S3	A loader fleet writing slices of one table at once; a day register; publication by fingerprint	loads of very large days finish in the window
S4	Products (link indexes, group totals, rollups, id lists) written with the data	aggregates and links without scanning
S5, S6	Each server owns a share of (table, day) slices on the ring; split searches and aggregates	N servers hold N × one server's working set
S7	Warm tier of memory-mapped column files of owned days	searches on hot days from memory
S8	Benchmark at scale in AWS	measured numbers replace the estimates of section 12

04

First-time setup, step by step

Twelve steps (0–11). Each: the script, the exact command, what you should see. Always --dry-run first, read it, then run

Setup at a glance: steps 0 to 11

0 Prerequisites

VPC, endpoints, EFS, KMS, ECS, ALB, SNS, secrets

1 Settings

`deploy/aws/env` from `env.example`

2 Buckets

`setup-buckets.sh`

3 IAM

`setup-iam.sh`

4 Prove commits safe

`check-conditional-writes.sh`

5 Storage tiers

`setup-lifecycle.sh`

6 EFS access points

`ecs/setup-efs.sh`

7 Cloud Map seeds

`ecs/setup-cloud-map.sh` (twice)

8 Images

`docker build` ×3, push to ECR

9 Tasks, services, schedules

`ecs/register.sh`

10 First load

`load-day.sh --date …`

11 Alarms and report

`setup-alarms.sh`, `report.sh`

Step 0: prerequisites (the platform team's templates)

- A **VPC** with private subnets in **three Availability Zones**
- A **gateway VPC endpoint for S3** on their route tables (lake traffic never leaves the VPC; the bucket policy requires it)
- **Interface endpoints** for ECR, CloudWatch Logs, Secrets Manager and STS
- An **EFS file system**: Regional, encrypted, **Elastic throughput**, a mount target per subnet
- A **KMS key** for the lake; its key policy allows the roles of step 3
- An **ECS cluster**
- An **Application Load Balancer** with a target group for the consoles on **port 17480**
- An **SNS topic** for alarms
- The **four secrets** of AWS_DEPLOYMENT.md §11.3 (token secret, session secret, cluster key, admin seed)

Who builds these? The bank's platform team, with its own CloudFormation or Terraform templates.
The Drishti scripts take their **ids** (subnets, file system, key ARN, cluster, target group, topic) from `deploy/aws/env`.

Run from a workstation or admin host with: AWS CLI v2, the repository, Java 21, and `drishti-console/.venv`.

Step 1: the settings file


```
cp deploy/aws/env.example deploy/aws/env        # git-ignored; holds account ids and names, never a secret
$EDITOR deploy/aws/env                          # replace every <placeholder>
cp deploy/aws/delta-lake/tables.example deploy/aws/delta-lake/tables.conf   # optional: your tables; set LAKE_TABLES_FILE
```

You should see: nothing yet. A script run with a placeholder left names it:
`setup-buckets.sh: LAKE_KMS_KEY_ARN is still a placeholder (<arn:aws:kms:…>) in deploy/aws/env`

The tables file: one line per table, seven fields:

```
trading/trade_endur     trade   config/packs/trading   trade-endur     {date}/endur/          -      PRODUCT_TYPE
```

Field	Example	Meaning
folder	`trading/trade_endur`	the table's folder under the lake prefix (domain / table)
kind	`trade`	the Drishti kind
pack	`config/packs/trading`	the pack whose layout (key, promoted columns) is used
connector	`trade-endur`	the pack's connector that reads this table
landing prefix	`{date}/endur/`	where the dumps are; `{date}` is the business date
business-date field	`-`	the field holding the date, or `-` for the day being loaded
partition column	`PRODUCT_TYPE`	the second partition key

Step 2: the buckets


```
deploy/aws/delta-lake/setup-buckets.sh --dry-run
deploy/aws/delta-lake/setup-buckets.sh
```

You should see each call as it runs, ending in `setup-buckets.sh: done: acme-drishti-prod-lake, acme-drishti-prod-landing, acme-drishti-prod-logs`

```
+ aws s3api create-bucket --bucket acme-drishti-prod-lake --object-ownership BucketOwnerEnforced \
      --create-bucket-configuration LocationConstraint=eu-west-2
+ aws s3api put-public-access-block --bucket acme-drishti-prod-lake --public-access-block-configuration BlockPublicAcls=true,…
+ aws s3api put-bucket-versioning --bucket acme-drishti-prod-lake --versioning-configuration Status=Enabled
+ aws s3api put-bucket-encryption --bucket acme-drishti-prod-lake --server-side-encryption-configuration file://…/enc-….json
+ aws s3api put-bucket-policy --bucket acme-drishti-prod-lake --policy file://…/policy-….json
…
```

- **Idempotent:** run again, it says `acme-drishti-prod-lake exists: kept, settings applied again` and puts the same settings
- **Three buckets:** `-lake` (every Delta table), `-landing` (dumps, 30 days), `-logs` (access logs, 400 days)
- **On the lake:** versioning on; SSE-KMS with S3 Bucket Keys; Block Public Access; bucket-owner-enforced ownership; TLS only; VPC endpoint only (plus `drishti-<env>-admin*` roles); access logs; request metrics on `lake/`; noncurrent versions expire after 14 days; incomplete multipart uploads abort after 2 days
- **`--dry-run`** prints each changing call as `+ aws …` and each read as `? aws …`; none is made

Step 3: who may do what (IAM)


```
deploy/aws/delta-lake/setup-iam.sh --dry-run
deploy/aws/delta-lake/setup-iam.sh
```

Role	Policy (deploy/aws/delta-lake/iam/)	May
drishti-<env>-server-task	server-policy.json	read the lake prefix (GET, LIST), decrypt; mount EFS through access points
drishti-<env>-loader-task	loader-policy.json	read the landing bucket; write data files and commits; **never delete** (an explicit deny)
drishti-<env>-maintenance-task	maintenance-policy.json	checkpoints, retention and vacuum: **the only role that deletes**; the lifecycle rules
drishti-<env>-execution	AWS AmazonECSTaskExecutionRolePolicy + drishti/<env>/* secrets	ECS pulls images, writes logs, injects secrets
drishti-<env>-scheduler	inline	EventBridge Scheduler runs the lake tasks

You should see `setup-iam.sh: done: three policies, five roles (…)`. A policy that exists gets a new default version.
Trust policies: `iam/ecs-tasks-trust.json` (task roles), `iam/scheduler-trust.json` (the scheduler role).

Step 4: prove that commits are safe


```
deploy/aws/delta-lake/check-conditional-writes.sh
```

You should see `check-conditional-writes.sh: ok: the second create was refused (412 PreconditionFailed): concurrent commits are safe`

- **What it does:** creates a test object with `If-None-Match: *`, then tries to create it again; S3 must refuse the second with **412**
- **Why it matters:** a Delta commit is the creation of `_delta_log/<version>.json`; two writers must never both create the same version (section 8.2)
- **Any other answer: stop.** An S3-compatible store or a proxy that drops the header cannot guarantee safe concurrent commits; then only one writer per table at a time (the scripts' per-table locks already give that if nothing else writes)
- **No DynamoDB lock table** is needed: deltalake 1.x writes commits with conditional PUT (its message: *delta-rs no longer supports S3DynamoDBLogStores*)

Step 5: storage tiers by business date


```
deploy/aws/delta-lake/setup-lifecycle.sh --print | head -20     # the rules, no AWS call
deploy/aws/delta-lake/setup-lifecycle.sh
```

You should see `setup-lifecycle.sh: 129 rules for acme-drishti-prod-lake` (three tables from 2019: one rule per table per month still moving, one per table per year already in Glacier IR, three base rules).

- **By date, not by age:** age is the time since an object was written, wrong for a restated or backfilled day; so one rule per table per business month, with transition **dates**
- **Standard → Standard-IA** when the month's last day is 90 days old (`TIER_IA_AFTER_DAYS`); **→ Glacier Instant Retrieval** at two years (`TIER_GLACIER_IR_AFTER_DAYS`)
- **Never** Glacier Flexible Retrieval or Deep Archive (hours to restore); **never** an S3 expiry of current lake objects
- `_delta_log/` never matches: commits and checkpoints stay in Standard; objects under **128 KB** stay where they are
- Rules exist **three months ahead**; `maintain.sh` rewrites them nightly; S3 allows **1,000 rules** and the script refuses more (about 30 tables over seven years)
- Details and the rule JSON: section 9.4

Step 6: EFS access points, one per node per path


```
deploy/aws/ecs/setup-efs.sh --dry-run
deploy/aws/ecs/setup-efs.sh
```

You should see one access point id per line (`fsap-0a1b…`) and `setup-efs.sh: 14 access points on fs-…` for three nodes.

Owner	Access points	Root (identical path inside the container)
each node (drishti-a, -b, -c)	`data`, `config`, `sutras`	`/drishti/<env>/<node>/opt/drishti/<path>` → `/opt/drishti/<path>`
consoles	`data`, `config`, `sutras`	`/drishti/<env>/consoles/opt/drishti/<path>`
loader	`data`	`/drishti/<env>/loader/opt/drishti/data` (scratch lakes, table locks)
maintenance	`data`	`/drishti/<env>/maintenance/opt/drishti/data`

- Every access point runs as **uid/gid 10001** (the image's user); 3 × 3 + 3 + 1 + 1 = **14**
- **Idempotent:** the access point's name is its client token, so running again returns the same ids
- The user's rule from AWS_DEPLOYMENT.md: every path a container writes maps to the **identical path** on EFS

Step 7: Cloud Map for Drishti*net seeds


```
deploy/aws/ecs/setup-cloud-map.sh        # once for the namespace (it takes a minute) …
deploy/aws/ecs/setup-cloud-map.sh        # … and again for one service per node
```

- **You should see** `namespace requested: it takes a minute; run this script again to create the services`, then one `create-service` per node.

- ECS registers each node's task under `drishti-a.drishti.local` with the **task's own address**
- The servers advertise `http://<task IP>:18480`, **never loopback** (Drishti*net refuses 127.0.0.1, localhost and 0.0.0.0 since 2.6.0)
- A new node finds the cluster through its seed's Cloud Map name, then learns the rest by gossip
- The loader's data-load signal (`DRISHTI_SIGNAL_URL`) also uses a node's Cloud Map name

Step 8: the images


```
./mvnw -q package -DskipTests
docker build -f deploy/docker/ecs/server.Dockerfile  -t "$ECR_REGISTRY/drishti-server-ecs:2.6.5" .
docker build -f deploy/docker/ecs/console.Dockerfile -t "$ECR_REGISTRY/drishti-console-ecs:2.6.5" .
docker build -f deploy/docker/ecs/loader.Dockerfile  -t "$ECR_REGISTRY/drishti-loader-ecs:2.6.5" .   # carries
deploy/aws/env
docker push …                                         # each of the three, after aws ecr get-login-password
```

Image	Built from	Entry point checks / role
drishti-server-ecs	deploy/docker/ecs/server.Dockerfile	server-entrypoint.sh: EFS mounted and writable, real IPv4 (never loopback), first-start node.yaml and peers; health check over curl
drishti-console-ecs	deploy/docker/ecs/console.Dockerfile	console-entrypoint.sh: the console, separate container from the server
drishti-loader-ecs	deploy/docker/ecs/loader.Dockerfile	loader-entrypoint.sh: EFS mounted and writable; runs the lake scripts; copies deploy/aws/ with your env (no secret)

- Loader tasks read the same settings: DRISHTI_AWS_ENV=/opt/drishti/app/deploy/aws/env; **rebuild the loader image when the settings change**
- The build context is the repository root; nothing is built or run on a developer laptop by the scripts

Step 9: task definitions, services and schedules


```
deploy/aws/ecs/register.sh --dry-run
deploy/aws/ecs/register.sh                          # --nodes --lake-jobs
deploy/aws/ecs/register.sh --consoles 4             # optional: a console service of its own (SUPER_SCALE.md §16)
```

- **You should see:** a task definition per node (`drishti-prod-drishti-a`, …), a service per node, the `loader`, `maintenance` and `report` task definitions and three schedules (`drishti-prod-load-day`, `-maintain`, `-report`), then `register.sh: done`.

- The node task sets **`DRISHTI_DELTA_ROOT=s3://<lake bucket>/lake`**, so every shipped Delta connector (`root: ${DRISHTI_DELTA_ROOT:…}`) reads the lake in S3 **with no edit**
- Templates: `deploy/aws/ecs/templates/` → `node-task`, `node-service`, `console-task`, `console-service`, `lake-task`, `schedule-target` (JSON with placeholders)
- `--cache nvme` (EC2 launch type) mounts the host's `/mnt/nvme/drishti/<node>` at `/opt/drishti/nvme` (section 10)
- Schedules: `LOAD_SCHEDULE` (22:30 Mon–Fri), `MAINTAIN_SCHEDULE` (Tue–Sat 02:30), `REPORT_SCHEDULE` (daily)

Step 10: the first load


```
deploy/aws/delta-lake/load-day.sh --dry-run --date 2026-10-20 --tables trading/trade_endur
deploy/aws/delta-lake/load-day.sh --date 2026-10-20 --tables trading/trade_endur
```

You should see the five steps of section 8.1, then
`lake publish: ok exit=0 published <n> file(s), <bytes> bytes, dates 2026-10-20 to s3://…/lake/trading/trade_endur version=0 (new table)`
and `load-day.sh: business date 2026-10-20: exit 0`.

On ECS the same runs as the `loader` task:

```
aws ecs run-task --task-definition drishti-prod-loader \
  --overrides '{"containerOverrides":[{"name":"loader","command":["deploy/aws/delta-lake/load-day.sh","--date","2026-10-20"]}]}' …
```

In the console: `TRD <id>` on 2026-10-20 opens the trade; **Admin → Health** shows `trade-endur UP (engine: native)`.

Step 11: alarms and the report


```
deploy/aws/delta-lake/setup-alarms.sh
deploy/aws/delta-lake/report.sh
```

- **You should see** `setup-alarms.sh: 17 alarms put` (four per table, two for the bucket and EFS, one per node), and the report of section 11.3.

- **Per table (×3 tables = 12):** `load-failed`, `not-loaded`, `load-slow`, `log-long`
- **Bucket and EFS (2):** `lake-s3-5xx`, `efs-io-limit`
- **Per node (×3):** `<node>-down`
- All notify `ALARM_SNS_TOPIC_ARN`; details in section 11.4
- `report.sh` is also scheduled daily (`REPORT_SCHEDULE`) and sends its numbers to CloudWatch

05
The layout

Buckets, prefixes, tables and partitions: what goes where in S3, and why

5.1 Buckets

Bucket	Holds	Settings (setup-buckets.sh)
`<org>-drishti-<env>-lake`	every Delta table: data files and `_delta_log`	versioning **on**; SSE-KMS with **S3 Bucket Keys** (one KMS call per object set, not per object); Block Public Access; objects owned by the bucket owner; **TLS only**; data reachable **only through the VPC endpoint** (and `drishti-<env>-admin*` roles); access logs; request metrics on `lake/`; **noncurrent versions expire after 14 days**; incomplete multipart uploads abort after 2 days
`<org>-drishti-<env>-landing`	the sources' dumps	the same protections; dumps expire after **30 days**
`<org>-drishti-<env>-logs`	S3 access logs	SSE-S3; expire after **400 days**

Versioning on, with a short noncurrent window. Delta keeps its own history (time travel back to the vacuum window), so versions are not for history: they are a **safety net**. A file deleted by mistake (a wrong vacuum, a script against the wrong prefix) stays recoverable for `NONCURRENT_KEEP_DAYS` (14). The lifecycle expires noncurrent versions after that, so storage does not double. (AWS_DEPLOYMENT.md §8.1 updated to match.)

5.2 Prefixes and tables


```
s3://acme-drishti-prod-lake/
└── lake/                    LAKE_PREFIX: DRISHTI_DELTA_ROOT = s3://…/lake
    ├── trading/             the domain (the connector's domain)
    │   ├── trade_endur/     one table per source system and kind
    │   │   ├── _delta_log/  commits, checkpoint every 10 commits,
    │   │   │                _last_checkpoint (S3 Standard, always)
    │   │   ├── business_date=2026-10-19/
    │   │   └── business_date=2026-10-20/
    │   │       ├── PRODUCT_TYPE=FX_FORWARD/part-00000-<uuid>.c000.snappy.parquet
    │   │       └── PRODUCT_TYPE=IRS_FIXFLOAT/part-00000-<uuid>.c000.snappy.parquet …
    │   ├── trade_calypso/ …
    │   └── trade_murex/ …
    ├── counterparty/ …      reference data: small tables, often mode: effective
    └── market/ …
```

Choice	Why
One table per source system and kind (`trade_endur`, `_calypso`, `_murex`, read as a union, `union: true`)	each source loads, restates and fails on its own; each log a third the size; the trading pack's three connectors already read them so
Domain as the first folder	matches the connector's `domain`; IAM and lifecycle rules can name a domain
Tables by year (`trade_endur/2026/`)	**planned (S2)**: until then one table holds every year and its checkpoint grows with live files

5.3 Partition keys and request-rate-friendly prefixes

Key	Why
`business_date` **(always first)**	every interactive read names a date, so a read opens one day's folder only; a load and a restatement replace one day; retention removes whole days; the lifecycle tiers by the folder's date
`PRODUCT_TYPE` **for trades (second key,** `--partition-by`**)**	spreads a day over several S3 prefixes (request rate); keeps each partition small enough for `lake build` to sort in one batch; lets a source restate one product type (`--mode overwrite-partitions`)
nothing finer	each partition column multiplies the folders and small files of a day; `lake build` allows four; **two for trades, one for everything else**

- **S3 scales each prefix** to ≥ 3,500 writes and 5,500 reads a second and splits busy prefixes by itself
- A day's reads spread over its **product-type folders**, and over **three tables** for trades
- Delta paths cannot be hashed (Drishti's writers name the folders), so the spread comes **from the partition keys**
- A `503 Slow Down` is retried by the native engine's AWS client with back-off and **counted in Health**; alarm `drishti-<env>-lake-s3-5xx` fires on a sustained rate

06
Keys

The id, the promoted columns and the statistics: what each kind of read needs from the table

6. Columns of every table

Column	Type	Content
`id`	string	the entity's id (`--key`, or the pack's `ingest.<kind>.key`: `TRADE_ID`)
`doc`	string	the whole JSON document as the source sent it
`business_date`	date	partition
partition columns	string	`PRODUCT_TYPE`
promoted columns	string or double	copies of the fields searches, lists, links and BI read: the pack's `layout.<kind>.columns`, plus `ingest.<kind>.flatten` (`NOTIONAL`, `INDEX_NAME`)

- Choosing what to promote is **a pack decision** (DELTA_CONNECTOR.md §4)
- Adding a promoted column later **changes the table's columns**: `lake publish` refuses a day whose columns differ: `the columns differ from the target's (new [...], missing [...], other type [...])` → runbook 13.4

6. What each read needs

Need	What it reads	So the table needs
One entity (`TRD MX-20000017`)	the day's id map (the `id` column of that day's files), then **one row group of one file**	rows **sorted by id** per file; small row groups (section 7)
History (`TRD MX-20000017 30`)	the snapshot's `business_date=` partitions, then one single read per day, eight at once	`business_date` as the partition; nothing else
Type-ahead	the newest day's ids (and `index.days` past days) from the id map, into the in-memory index	`id`; nothing else
Search, pick lists, derived kinds, impact, links	the day's column set: `id` and every promoted column, **never doc**	every field a search filters, sorts or shows **promoted**; else documents are read up to `drishti.search.max-scan` and the answer says `partial`
File skipping	per-file min/max in the log; per-row-group min/max in the footer	`delta.dataSkippingStatsColumns` = `id`, `business_date`, promoted columns, **never doc**; `lake build` sets it, `lake publish` carries it to S3

07
Files

Sizes, row groups, compression, statistics, checkpoints, retention, sorting by id, and one commit per day per table

7.1 The numbers

Setting	Value	Set by	Why
Rows per file	`layout.<kind>.file-rows` (**250,000**)	the pack	few files per day, each large enough for S3 (hundreds of MB)
Row group	**1 MB** (`LAKE_ROW_GROUP_KB=1024` → `DRISHTI_LAKE_ROW_GROUP_KB`; writer default 256 KB)	`deploy/aws/env`	a single read fetches one row group; each S3 GET costs 10–30 ms to first byte, so 1 MB balances bytes vs GETs; page index narrows it (`DRISHTI_LAKE_PAGE_KB`, 64)
Compression	**Snappy**	the writer	the native engine also reads ZSTD, GZIP, LZ4_RAW
Statistics	`id`, `business_date`, promoted columns	`lake build`	file and row-group skipping without documents in the log
Checkpoint	every **10 commits** (`delta.checkpointInterval`) and after every maintenance run	`lake publish`, `lake maintain`	a reader replays the last checkpoint plus a few commits
Log retention	**30 days** (`delta.logRetentionDuration`)	`lake publish`	older commit files (before a checkpoint) removed by `lake maintain`
Removed files kept	**7 days** (`delta.deletedFileRetentionDuration`; `LAKE_VACUUM_HOURS` 168)	`lake publish`, `maintain.sh`	a reader in flight and *known at* reach back a week

7.2 Sorted by id

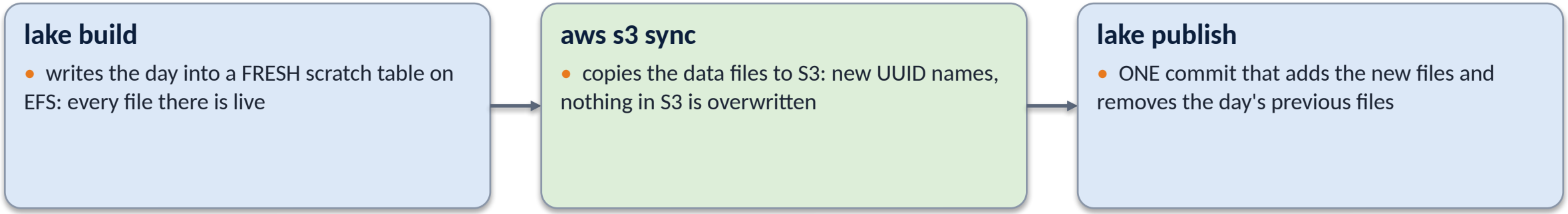
- A file holds a **contiguous id range** and each row group a smaller one, so a read of one id **skips every other row group** by its statistics
- `lake build` sorts **each partition (date × product type) that fits in one batch** (`--batch-rows`, `LAKE_BATCH_ROWS`, 250,000)
- A larger partition is written **in the dumps' order**; `load-day.sh` prints a warning for each such partition
- A table written elsewhere (Spark, Databricks): `repartitionByRange(…, "id").sortWithinPartitions("id")`

Planned (SUPER_SCALE S1): range maps from the log's per-file id ranges instead of id maps, and Parquet Bloom filters on `id`.

Three remedies, in order of preference

1. **Ask the source to export sorted by id.** The dump's order becomes the file's order: sorted at any size, no memory cost.
2. **Partition finer** (`PRODUCT_TYPE`, or a book or desk column) so each partition fits a batch.
3. **Raise LAKE_BATCH_ROWS** with the loader's memory: about rows × average document size × 3. 250,000 trades of 10 KB need about 8 GB; the loader task has 30 GB.

7.3 One commit per day per table



- A reader sees **the old day or the new day, never a mixture**
- The previous version stays readable (**time travel, *known at***) until vacuum (`LAKE_VACUUM_HOURS`, 168 h)
- One commit per table per day ⇒ about **1,500 commits a year per table** including maintenance (section 12.1)
- `lake publish` checks that **every file is in S3 before committing**, and refuses a day whose columns differ from the table's

08
Loading

The daily load, why concurrent commits are safe, backfill and restatement

8.1 The daily load (load-day.sh)


```
EventBridge Scheduler (LOAD_SCHEDULE, LOAD_SCHEDULE_ZONE)
  └─► ECS RunTask drishti-<env>-loader ─► loader-entrypoint.sh (EFS mounted? writable?) ─► load-day.sh
        for each table of LAKE_TABLES_FILE:
          flock  /opt/drishti/data/lake-locks/<domain>--<table>.lock   (another load, backfill or maintenance: exit 3)
          aws s3 sync   s3://landing/<date>/<source>/  →  scratch/in/<table>/
          drishti.py lake build   --from scratch/in/<table> -r --kind trade --pack config/packs/trading --connector trade-endur
                                  --lake scratch/lake --domain trading --table trade_endur --mode overwrite-dates
                                  --batch-rows 250000 --as-of <date> --partition-by PRODUCT_TYPE --no-stamp --summary …
          aws s3 sync   scratch/lake/trading/trade_endur/  →  s3://lake/lake/trading/trade_endur/   --exclude "_delta_log/*"
          drishti.py lake publish --from scratch/lake/trading/trade_endur --to s3://…/lake/trading/trade_endur
                                  --dates <date> --mode overwrite-dates --storage-option AWS_REGION=<region> [--signal …]
          CloudWatch Drishti/Lake: LoadSucceeded, PublishedBusinessDate, LoadSeconds
        scratch removed (always)
```

Exit	Meaning
0	every table published
2	published with rejected documents (the summary says which)
1	a table failed: its S3 table is unchanged
3	a table was locked

The date defaults to today in `LOAD_SCHEDULE_ZONE`, the zone the schedule runs in: an evening load in New York is already the next day in UTC.

8.1 The data-load signal

- **With** `DRISHTI_SIGNAL_URL` (a node's Cloud Map name) and `DRISHTI_TOKEN_SECRET_ID` (an API token with `loads:write` in Secrets Manager, read into a **0600 file** for the run)
  - `lake publish` sends `POST /api/v1/packs/{pack}/loads` **per date**
  - the server refreshes, verifies and alerts as for any load (DATA_LOADS.md)
- **Without it**, servers see the new table version at their next refresh (`refresh-seconds`, **10**)
- Either way readers switch at the commit; the signal only makes it immediate and runs the load's expectations and alerts

8.2 Why concurrent commits are safe

Writer	How it commits on S3
`lake publish` and `lake maintain` (Python `deltalake` 1.6.6, in the venv and the loader image)	a **conditional PUT** of the commit file (put-if-absent, `If-None-Match: *`): S3 refuses the second writer with **412**, which re-reads the log and retries (`--commit-retries`, 20). deltalake 1.x no longer has the DynamoDB lock, so **no lock table**. `AWS_S3_ALLOW_UNSAFE_RENAME=true` is **refused** by `lake publish`
Drishti's Java writer (`lake build`)	local disk and EFS only (a hard link of a finished commit file); **never S3**
Drishti's native Iceberg writer (`drishti-lakeio`)	the same conditional PUT, with a start-up probe

Two layers of safety. `check-conditional-writes.sh` proves the bucket refuses a second create. On top of it, the scripts take **a lock per table on EFS** (`flock`, NFSv4 byte-range locks, released if a task dies), so a load, a backfill and maintenance of one table **never overlap**. Loads of **different** tables run side by side.

8.3 Backfill and 8.4 restatement


```
deploy/aws/delta-lake/backfill.sh --from 2026-01-02 --to 2026-10-16 --skip-missing          # weekdays; dates without dumps skipped
deploy/aws/delta-lake/backfill.sh --from 2025-01-02 --to 2025-12-31 --from-dir /opt/drishti/data/history   # an export on EFS
```

- **Backfill**
- **Oldest date first**, one `load-day.sh` per date
- **Stops at the first date that fails** and prints the command that resumes from there
- Re-running a date replaces it: a backfill can **always be run again**
- Large backfill: several loader tasks side by side **on different tables** (`--tables`); the per-table lock keeps two on one table apart

A source re-sent	Run
some whole days	`backfill.sh --dates 2026-10-14,2026-10-15`
one product type of a day	`load-day.sh --date 2026-10-15 --tables trading/trade_endur --mode overwrite-partitions`

Restatement: readers switch to the restated day at the commit; *known at* before the commit still shows the day as it was, for `LAKE_VACUUM_HOURS`. Data files of an old restated day move to **its** storage tier at the next lifecycle run (rules are by date, not object age), so a restatement of 2024 lands in Glacier IR, not Standard.

09
Maintenance

Checkpoints, retention, vacuum, compaction and the lifecycle rules: what runs nightly and why

9.1 What maintain.sh runs (nightly)

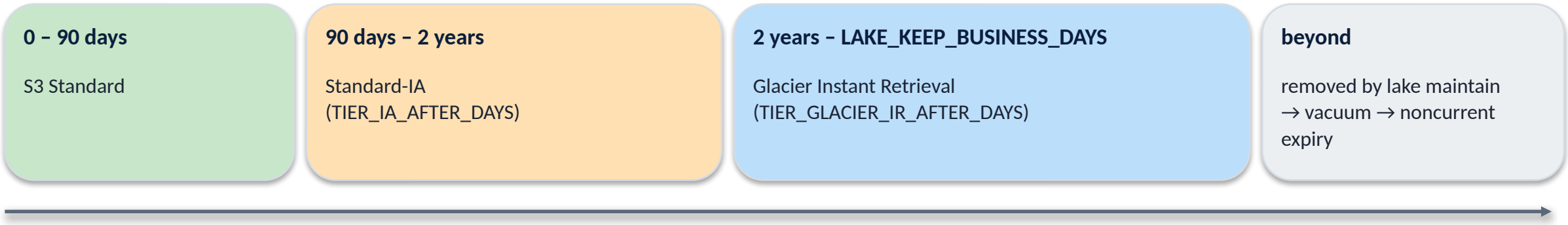

```
drishti.py lake maintain --config <lake-maintenance.s3.yaml rendered> --once --force-foreign --tables trading/trade_endur
```

Nightly (`MAINTAIN_SCHEDULE`, **Tuesday to Saturday 02:30**), as the `maintenance` task, per table under the table's lock; then `setup-lifecycle.sh`. Settings: `deploy/aws/delta-lake/lake-maintenance.s3.yaml`

Key	Value	What lake maintain does
`keep-business-days`	`LAKE_KEEP_BUSINESS_DAYS` (**1800**, about seven years)	removes business dates older than that, counted back from today: a delete by partition, one commit; refuses to remove more than `max-drop-share` (**0.5**) of a table
`checkpoint`	true	writes a checkpoint and removes commit files past `delta.logRetentionDuration`
`vacuum-hours`	`LAKE_VACUUM_HOURS` (**168**)	deletes files no version uses, older than a week; S3 keeps them **14 more days** as noncurrent versions

--force-foreign: `lake maintain` maintains only tables `lake build` built in that location and treats every `s3://` table as foreign. The S3 tables are Drishti's (`lake publish` wrote them from `lake build`'s output), so the script overrides the rule; `lake maintain` prints its warning and marks the runs *forced*.

9.2 Retention and the tiers together



- **Never** Glacier Flexible Retrieval or Deep Archive: their objects need a **restore of hours** before a read, and Drishti reads any day in milliseconds
- **Never** an S3 expiry of current lake objects: removing a file a table version still names **breaks the table** (DELTA_CONNECTOR.md §10)
- **Only lake maintain removes table files**; S3 then keeps the noncurrent versions 14 days

9.3 Compaction: why it is off for S3

- Each day **arrives laid out**: files of `file-rows`, sorted by id, one commit ⇒ the S3 tables need **no compaction**
- A plain Delta compaction would **merge files out of id order** (single-entity reads would then touch many row groups)
- `lake maintain` re-sorts (`relayout`) only tables whose folder is the **kind's name** (`trading/trade`)
- For the trading pack's **per-source tables** (`trade_endur`) it finds no layout and would run a plain compaction
- So **`compact: false`** for S3; a day that needs re-laying is **restated** instead (runbook 13.4)

Open finding (fix run 2.6.5 QA): `tools/lake/maintain.py` looks up a table's layout by its folder name, but layouts are keyed by kind. Fixing that lookup (and checking `relayout` with `PRODUCT_TYPE` partitions) would let maintenance re-sort per-source tables safely.

9.4 The lifecycle rules (setup-lifecycle.sh)


```
{"ID": "tier-trading-trade_endur-2026-03", "Status": "Enabled",
 "Filter": {"And": {"Prefix": "lake/trading/trade_endur/business_date=2026-03",
                    "ObjectSizeGreaterThan": 131072}},
 "Transitions": [{"Date": "2026-06-29T00:00:00Z", "StorageClass": "STANDARD_IA"},
                 {"Date": "2028-03-30T00:00:00Z", "StorageClass": "GLACIER_IR"}]}
```

- Rules match **prefixes**; move on a **date** or an **age**
- Age is wrong for restated/backfilled days ⇒ **one rule per table per business month, with dates**
- A whole year already past two years is **one rule**

- A month's files move to **Standard-IA** when the month's last day is **90 days** old, and to **Glacier IR at two years**
- Rules exist **three months ahead**; `maintain.sh` rewrites them nightly
- `_delta_log/` never matches: **commits and checkpoints stay in Standard**
- Objects **under 128 KB** stay where they are (moving them costs more than it saves)
- S3 allows **1,000 rules**; the script refuses more (≈ 30 tables over seven years)

Costs to know: Standard-IA keeps an object at least **30 days**, Glacier IR **90 days**; a restatement of a tiered day pays the rest of that minimum on the files it replaces. Reads from Standard-IA ≈ **USD 0.01/GB**, from Glacier IR ≈ **USD 0.03/GB**.

10

Caches: EFS or local NVMe

The small, bounded structures each server keeps, where they live, and the one decision for the product owner

10. What each server caches

Cache	Where	Bounded by	After a restart
file list (snapshot) of each table	heap	the table's live files	re-read from the checkpoint
Parquet footers	heap (`-Ddrishti.parquet.footer-cache-groups`, 50,000 row groups)	row groups	re-read on first use (one ranged GET)
id maps of days	heap (`id-map-mb`, 20 % of heap, shared) and disk (`id-map-disk.dir`)	memory; one file per table version and date	**memory-mapped from disk: no scan**
column sets of days	heap (`columns-cache-mb`)	memory	re-read (the newest day in the background)
documents	heap (`doc-cache-mb`) and RocksDB on disk (`doc-disk-cache.dir`, `max-gb` 1, TTL 12 h, cleared daily 02:00)	size, age	**warm from disk**
type-ahead	heap (newest day; `index.days` more, `index.storage: mmap`)	days	rebuilt from id maps

The disk caches default to `<data dir>/cache/<connector>/…`, which on ECS is `/opt/drishti/data/cache/…` on **EFS**.

10. EFS or local NVMe for the disk caches

	EFS (default)	Local NVMe (instance store)
Latency	≈ 1 ms an operation (NFS); page cache serves repeated reads of memory-mapped id maps	tens of microseconds
Throughput mode	**Elastic**: pays per GB read and written; Provisioned only for a steady high rate	the disk's own
After a task moves host or zone	**warm**: same folders	**empty**: rebuilt as users read (correct, slower first reads)
Launch type	Fargate or EC2	**EC2 only** (`m6id`, `r6id`, `i4i`)
Cost	EFS Standard ≈ USD 0.30/GB-month + Elastic charges	included in the instance
Identical-path rule	satisfied	**not**: the cache path is on the host

```
# config/connectors/trade-endur.yaml
  id-map-disk.dir:

/opt/drishti/nvme/cache/trade-endur/idmaps
  doc-disk-cache.dir:

/opt/drishti/nvme/cache/trade-endur/docs
  doc-disk-cache.max-gb: "50"
```

To use NVMe: `register.sh --nodes --cache nvme` (EC2; host `/mnt/nvme/drishti/<node>` → `/opt/drishti/nvme`), then point the caches there.

Recommendation: start on **EFS** (warm after any restart, no host affinity). Move the **document cache** to NVMe if Admin → Caches shows cold reads dominate and EFS latency shows in `p99Ms`.
Decision needed: NVMe breaks the identical-path rule; allowed only for rebuildable caches, by the product owner's decision.

11
Staying fast past 200 TB

What never scans the whole lake, spreading work over Drishti*net, the numbers to watch and the alarms

11.1 What never scans the whole lake

Request	Reads	Grows with history?
Open one entity on a date	the table's snapshot (in memory, refreshed from the last checkpoint every 10 s), the day's id map, **one row group of one file** (one ranged GET, ≈ 1 MB)	**no**
History of one entity over N days	the snapshot's `business_date=` partitions, then one single read per day held, eight at once	with N, **not with the lake**
Type-ahead	the newest day's ids in memory	**no**
Search, pick list, derived kind, impact on a date	that day's column set (`id` + promoted, never documents); files and row groups skipped by statistics	**no**
Load, restatement	one day's files and one commit	**no**
Retention	a delete by partition (the log only), one commit	**no**
Vacuum (weekly is enough)	a listing of the table's prefix	with the table's file count (≈ 1,000 objects per LIST)
Questions over all of history	not interactive: **Athena, Trino or Spark** on the same tables	—

11.2 Spreading the work over Drishti*net

- **Built today** (DRISHTI_NET.md §10):
- Nodes whose connectors read **different tables** of a kind are **shards**: a search asks every one; a read goes to the node that holds the table
- Give each node (or pair) its own tables: `drishti-a`, `drishti-b` read `trade_endur`; `drishti-c`, `drishti-d` read `trade_calypso` and `trade_murex`; each node caches only its tables
- Nodes that read **the same table** are **replicas**: a search asks one; a forwarded read goes to the same node for the same entity (stable hash of its id), so its caches stay warm
- Every node can read every table from S3: **a failed node costs speed, never answers**

drishti-a

- trade_endur

drishti-b

- trade_endur (replica)

drishti-c

- trade_calypso, trade_murex

drishti-d

- trade_calypso, trade_murex

S3 lake

- every node can read every table

Planned (S5, S6): ownership of (table, day, group) slices on the ring, so N servers on one table hold N × one server's working set, and searches split over the owners.

11.3 The numbers to watch: report.sh


```
bucket acme-drishti-prod-lake (CloudWatch daily storage metrics):
  StandardStorage                 10412.3 GB
  StandardIAStorage               46120.8 GB
  GlacierInstantRetrievalStorage 143998.1 GB
  objects                        672310

table                  version  checkpoint  behind   dates       newest   files GB(newest)    <32MB
trading/trade_endur       4512        4510       2    1764   2026-10-20     128      41.37        0
```

`report.sh` is scheduled daily (`REPORT_SCHEDULE`), prints these and sends them to CloudWatch. Illustrative numbers for a lake of about 200 TB.

11.3 The numbers to watch: healthy values and what to do

Number	Healthy	If not
`behind` (commits since the checkpoint)	under 10	a checkpoint failed: run `maintain.sh --tables …`; alarm `log-long` above 50
files of the newest day, and `<32MB`	≈ rows ÷ `file-rows` per partition; small files 0	many small files: the source sends many tiny dumps, or partitions are too fine (5.3)
live files per table	under ≈ 100,000	the checkpoint and every snapshot refresh grow with it: split the table (by source, or by year when S2 lands)
S3 5xx	rare	`503 Slow Down`: reads concentrated on one prefix; spread with partitions; alarm `lake-s3-5xx`
Admin → Health per connector: `p50Ms`, `p99Ms`, cold reads	single read p95 **< 300 ms** on newest days, **< 1 s** on any day	footers and id maps not warm (heap too small: Admin → Caches), or EFS latency (10)
Admin → Caches: the one memory budget	under its cap	raise the heap (**8 GB per million entities a day**), or shard tables over more nodes (11.2)
`LoadSeconds`	well inside the window	dumps are large: more tables in parallel (one loader task per table), or larger loader tasks

11.4 The CloudWatch alarms (setup-alarms.sh)

Alarm	Fires when
`drishti-<env>-<table>-load-failed`	a load of the table failed (nothing of it was published)
`drishti-<env>-<table>-not-loaded`	no successful load for **three days** running (a weekend is two)
`drishti-<env>-<table>-load-slow`	a load took longer than `ALARM_PUBLISH_DEADLINE_HOURS`
`drishti-<env>-<table>-log-long`	readers replay more than `ALARM_COMMITS_SINCE_CHECKPOINT` commits
`drishti-<env>-lake-s3-5xx`	more than `ALARM_S3_5XX_PER_5MIN` server errors in 5 minutes, twice in 15
`drishti-<env>-efs-io-limit`	EFS `PercentIOLimit` above **80 %**
`drishti-<env>-<node>-down`	a node's service has no running task for **5 minutes**

All notify `ALARM_SNS_TOPIC_ARN`. The AWS_DEPLOYMENT.md alarms (heap, Drishti*net member dead, backup failed) come with the platform's Prometheus or CloudWatch agent.

12

Sizing and cost

What 200 TB is, the deployment that serves it, and the monthly bill (list prices, one Region, estimates)

12.1 What 200 TB is (seven years of Parquet)

	Value
Per business day (252 a year)	about **113 GB** of Parquet; about **100 million trade documents** at about 1.1 KB each compressed (Snappy)
Files per day	about **350** (330 MB each, 250,000 rows) over all tables
Live files after seven years	about **620,000** over all tables; with three trade tables ≈ **200,000 each**: above the 100,000 of 11.3, so **tables by year (S2) or more tables matter before the seventh year**
Commits	one per table per day, plus maintenance: about **1,500 a year per table**

Estimates, to be replaced by the measurements of SUPER_SCALE step S8.

12.2 The deployment

Part	Size	Count
Server tasks	4 vCPU and 12 GB for the server (**8 GB heap**) in a 6 vCPU, 16 GB task with its console; larger nodes (16 vCPU, 64 GB) at SUPER_SCALE volumes	**3 to 6**, sharded by table (11.2)
Loader task	4 vCPU, **30 GB**, 21 GB ephemeral	one per table at load time, minutes to an hour a day
Maintenance and report tasks	the same image and size	nightly, minutes
EFS	each node's state and caches: 50–300 GB; loader scratch: one day of one table at a time (the day's dumps plus its Parquet)	one file system

12.3 Monthly cost (list prices, one Region, estimates)

Item	Basis	About
S3 storage, 200 TB tiered	10 TB Standard (USD 23/TB), 46 TB Standard-IA (12.5), 144 TB Glacier IR (4)	**USD 1,400**
S3 storage if all were Standard	200 TB × 23	USD 4,600
S3 requests	250 reads a second, two GETs each, USD 0.40 per million; loads and lists	USD 500–700
Retrievals from IA and Glacier IR	month-ends, audits, history	tens to a few hundred
Versioning overhead	14 days of replaced and vacuumed files	a few % of storage
Server nodes	3 tasks of 6 vCPU, 16 GB on Fargate, always on	USD 650–750
Loader, maintenance	about an hour a day	USD 50–150
EFS	1 TB Standard, Elastic throughput	USD 300 + throughput
CloudWatch, Secrets Manager, Cloud Map, endpoints, load balancer		USD 200–400

For comparison: the same 200 TB on EFS: **USD 60,000 a month** in Standard; about USD 3,200 in Infrequent Access plus USD 0.03 per GB read.

13
Runbooks

The daily load, a backfill, a restatement, a slow table, and disaster recovery

13.1 The daily load

Check	How
It ran	CloudWatch Logs `/drishti/<env>`, stream prefix `loader`; `LoadSucceeded` 1 per table
It published	`report.sh --tables <table>`: `newest` is today; in the console, `TRD <id>` on today's date
Exit 2 (rejects)	the build summary in the log names the reasons; fix the source and restate the day (13.3)
Exit 1	the log's line `lake build failed` or `lake publish failed` says why; the S3 table is unchanged; fix and run `load-day.sh --date <d> --tables <t>`
Exit 3	maintenance or a backfill held the table's lock; the next run loads it, or run it by hand after

13.2 A backfill and 13.3 a restatement

13.2 A backfill

1. `backfill.sh --dry-run --from … --to … --tables <t>`: check the dates and commands
2. Run it as a loader task per table (`aws ecs run-task … --overrides` with the command `deploy/aws/delta-lake/backfill.sh …`)
3. It stops at a failing date with the resume command; fix and resume
4. `setup-lifecycle.sh` (or the next `maintain.sh`) moves the backfilled years to their tiers

13.3 A restatement

1. The source re-sends the dumps under the landing prefix of the day
2. `backfill.sh --dates <d>` (whole day) or `load-day.sh --date <d> --tables <t> --mode overwrite-partitions`
3. Readers switch at the commit; the data-load signal (if set) refreshes servers at once

13.4 A slow table

Symptom	Cause	Fix
first read of a day slow, then fast	id map not yet on disk; footers cold	normal; `warm-dates` (3) warms the newest days at start
every read of a day slow	the day is not sorted by id (load warning *written in the dumps' order*), or row groups are large	restate the day after fixing the order (7.2)
searches slow or `partial`	a field the search uses is not promoted	promote it in the pack; the table's columns change so publish refuses the next day: write the new layout into a new folder (`trade_endur_v2`) with `backfill.sh` (its tables line pointing there), then switch the connector's `table.trade`
the snapshot refresh slow (Health)	many live files or a long log	`report.sh`: `behind` and file counts; run `maintain.sh`; split the table
`503 Slow Down`	one prefix overloaded	partition by `PRODUCT_TYPE` (or finer for that table); S3 scales a prefix within minutes

13.5 Disaster recovery

Event	How to recover
A table damaged (a wrong commit)	Delta time travel: the previous version is readable until vacuum; restate the affected days from the landing bucket (30 days) or from versions
Files deleted by mistake	S3 versioning: restore the noncurrent versions within `NONCURRENT_KEEP_DAYS` (`aws s3api list-object-versions`, `copy-object` of the version)
A Region lost	S3 Cross-Region Replication of the lake bucket with Replication Time Control (most objects within 15 minutes), EFS replication and ECR replication. Replication copies objects one by one, so a commit can arrive before a data file it names: a read of that day in the recovery Region fails (Health `DEGRADED … a truncated or missing Parquet file`) until the file arrives; other days unaffected. **Fail over after the backlog** (`ReplicationLatency`, `OperationsPendingReplication`) **is empty**
Node state lost	AWS Backup of EFS; caches rebuild from the lake by themselves

14
The scripts

Every file under deploy/, what it does, and the conventions every script follows

14. deploy/: the files


```
deploy/
├── docker/                       compose.yaml, compose.data.yaml, server.Dockerfile, console.Dockerfile (local and Docker hosts)
│   └── ecs/                      server.Dockerfile, server-entrypoint.sh, console.Dockerfile, console-entrypoint.sh,
│                                 loader.Dockerfile, loader-entrypoint.sh
├── aws/
│   ├── env.example               every setting; copy to deploy/aws/env (git-ignored)
│   ├── lib/common.sh             --help, --dry-run, --env, the printed AWS calls, templates
│   ├── delta-lake/
│   │   ├── setup-buckets.sh      step 2        ├── load-day.sh        8.1
│   │   ├── setup-iam.sh          step 3        ├── backfill.sh        8.3, 8.4
│   │   ├── check-conditional-writes.sh  step 4 ├── maintain.sh        9
│   │   ├── setup-lifecycle.sh    step 5, 9.4   ├── report.sh          11.3
│   │   ├── setup-alarms.sh       step 11       ├── lake-maintenance.s3.yaml
│   │   ├── tables.example        the tables    └── iam/{server,loader,maintenance}-policy.json, ecs-tasks-trust.json, scheduler-trust.json
│   └── ecs/
│       ├── setup-efs.sh          step 6        ├── register.sh        step 9
│       ├── setup-cloud-map.sh    step 7        └── templates/{node-task,node-service,console-task,console-service,lake-task,schedule-target}.json
├── grafana/                      the Grafana dashboard
└── lake-maintenance.yaml         lake maintain's example for a local lake (the S3 one is delta-lake/lake-maintenance.s3.yaml)
```

New tool in the repository: `tools/lake/publish.py` → `drishti.py lake publish` (TOOLS.md §4.11), tested by `tools/lake/test_publish.py`.

14. Conventions every script follows

- **bash** with `set -euo pipefail` and the licence header
- **AWS CLI v2 and nothing else**; Python only through `tools/drishti.py`
- **`--help`** on every script
- **`--dry-run`**: each changing call printed as `+ aws …`, each read as `? aws …`, **none made**
- **Settings from one file**: `deploy/aws/env` (from `env.example`; placeholders only; git-ignored), or `--env <file>`
- **Safe to run again** (idempotent): existing buckets, policies, access points and rules are kept and their settings applied again
- **A placeholder left** stops the script and names it
- **Locks:** load, backfill and maintenance take the table's `flock` on EFS; a held lock exits 3
- **Tested:** `tools/test_deploy_aws.py` runs every script with `--help` and `--dry-run` against a fake settings file and an `aws` that fails if called, renders every template to JSON, checks the lifecycle rules, and checks `env` is git-ignored (7 tests); both test files run in the drill

15

Limits and what comes next

What is not yet proven, known limits, and the decisions to take

15. Limits and what comes next

- **Not yet run in AWS.** The first real run, in a development account, follows sections 4 and 13 and records *what you should see*; AWS_DEPLOYMENT.md steps **A1** (images built, read-only-root test) and **A4** (EFS measurements) go with it
- **A bucket without conditional writes** (`check-conditional-writes.sh` exit 1: an S3-compatible store, or a proxy that drops the header): only one writer per table at a time; the scripts' per-table locks already give that if nothing else writes the tables
- **lake build sorts a partition only when it fits a batch** (7.2)
- **One table holds every year** until tables by year (S2): watch live files per table (11.3)
- **Maintenance cannot re-sort per-source tables** (9.3): `compact` stays off; restate instead
- **Planned:** ownership of slices, range maps, Bloom filters, products, the loader fleet (section 3)

Decisions and actions for the product owner

#	Decision or action	Recommendation
1	Disk caches on **local NVMe** (faster, but breaks the identical-path rule) or **EFS** (warm after restarts, keeps the rule)	Start on EFS; allow NVMe for rebuildable caches only, as a documented exception, when `p99Ms` shows EFS latency
2	First real run in a **development AWS account** (steps 0–11, runbooks 13.1–13.5)	Do it before any production commitment; record the outputs in the guide
3	Fix `lake maintain`'s layout lookup for per-source tables	In the 2.6.5 QA fix run; then compaction can be re-enabled safely
4	Tables by year (SUPER_SCALE S2)	Before about year five, when live files per trade table near 100,000
5	Range maps and Bloom filters (S1); slice ownership (S5, S6)	When the per-day id maps or one server's working set become the limit

Related documents

Document	What it holds
`docs/architecture/AWS_DELTA_LAKE.md`	this deck's source: every section, command and number
`docs/architecture/AWS_DEPLOYMENT.md`	ECS, EFS and the identical-path rule; secrets; images; DR
`docs/architecture/SUPER_SCALE.md`	500 GB a day for seven years; steps S1–S8
`docs/connectors/DELTA_CONNECTOR.md`	how Drishti reads a Delta table; layout in a pack; memory
`docs/admin/LAKE_MAINTENANCE.md`	`lake maintain`: retention, checkpoint, vacuum, relayout
`docs/guides/TOOLS.md`	`lake build`, `lake publish` (§4.11)
`docs/architecture/DRISHTI_NET.md`	several servers: shards, replicas, routing
`docs/guides/DATA_LOADS.md`	the data-load signal, expectations and alerts